OCP GLOBAL SUMMIT 2026 • ENSCO

用树莓派做时间服务器集群

Wil Myrick, Ph.D.（ENSCO 首席技术专家）

OTS-C：用 COTS 硬件与开源软件构建边缘时间韧性

时长 11 分 4 秒

来源 youtube.com/watch?v=W28mn4ihGs0

用树莓派计算模块做时间服务器集群

> **整理说明**　本文是 OCP Global Summit 2026 一场 11 分 4 秒演讲的图文实录。体例为：每页幻灯片 → 该页停留期间的**完整中文译文**（逐句对应，不删减、不概括）→ 英文原句 → 图下一句小结。分节按幻灯片切换顺序，标题不含时间戳。

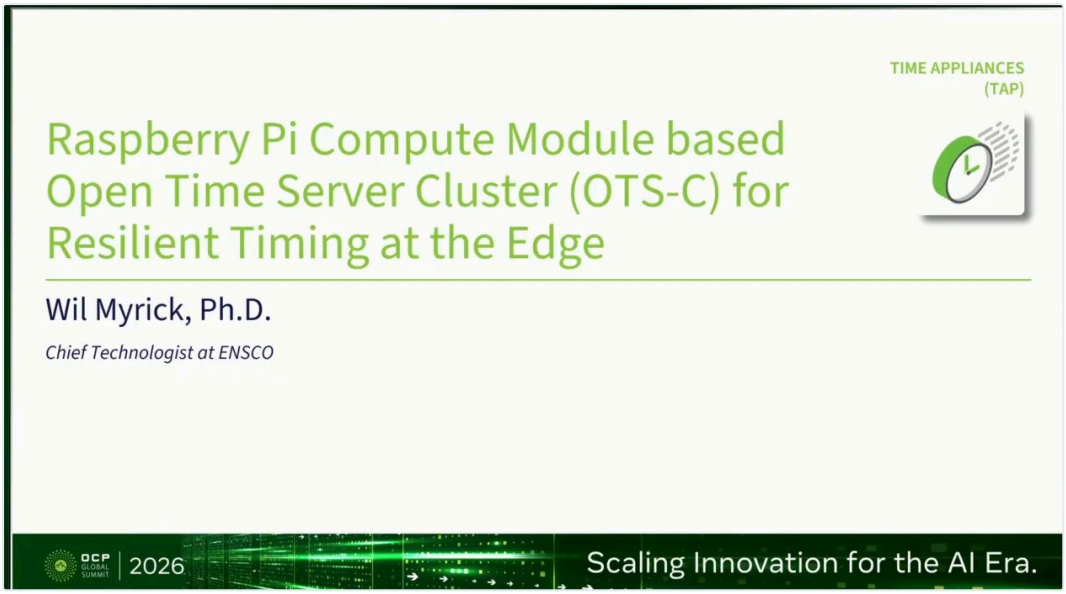
讲者　**Wil Myrick, Ph.D.** — Chief Technologist, ENSCO（首席技术专家）

这场演讲的位置　OCP 时间设备项目（TAP）此前的工作集中在"单台开放时间服务器"：一块时间卡、一台机器、一个高精度时钟源。Myrick 换了一个问法——如果不追求单点精度，而是把六台廉价机器组成一个**时间集群**，用软件把六路时钟合成一路，能不能在成本和韧性上同时占优？他用六块树莓派 CM4、六个 GNSS 接收机和开源软件 Chrony 搭了一台原型机来回答。

关于自动字幕的更正

字幕原文	更正为	说明
OTC / OTSC	OTS-C	Open Time Server – Cluster
CM 4s / CM uh 4s	CM4	Raspberry Pi Compute Module 4
Dr. Madnick	Dr. Myrick	讲者姓氏
open time card mini	Open Time Card Mini	TAP 的开源时间卡
PNT	PNT	Positioning, Navigation and Timing（定位、导航与授时）

01　开场：这次不谈时间卡，谈时间集群



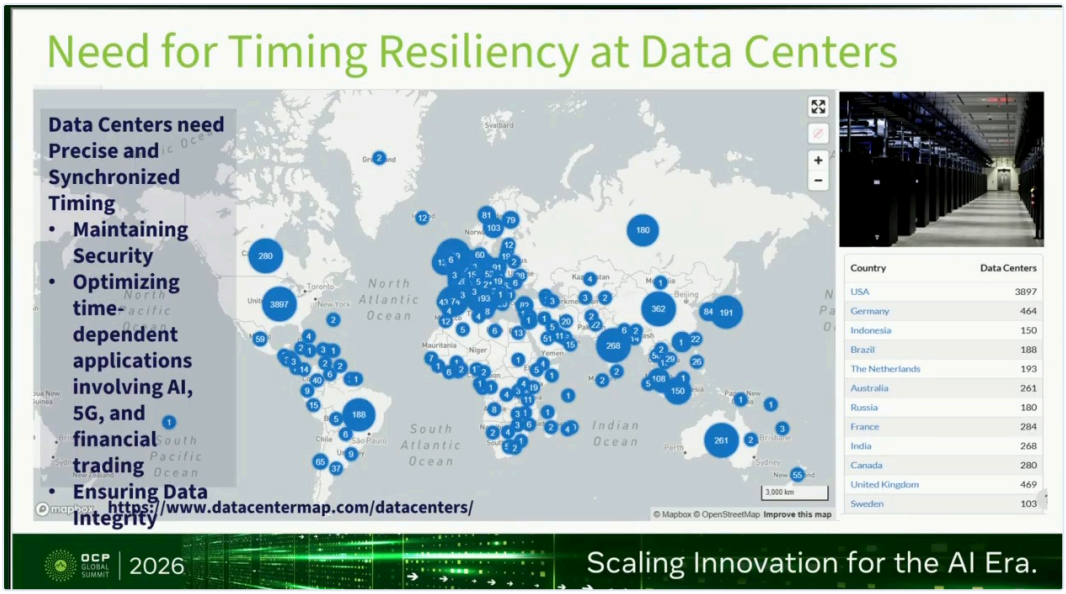
Raspberry Pi Compute Module based Open Time Server Cluster (OTS-C) for Resilient Timing at the Edge，Wil Myrick 博士（ENSCO 首席技术专家）

非常感谢，谢谢介绍。很高兴和大家在一起，很高兴今天下午能来到这里。要感谢 Ahmad，感谢他一直以来在推动 PTP 这件事上做的所有工作和努力。今天我想和大家谈的主要是延续"韧性"这个主题。我觉得两场演讲之前那一场讲得很好，他们讲了韧性、为什么需要韧性、韧性的重要性，特别是围绕时间卡来讲。而这一次，我要带来的是一些技术，而不是时间卡。我要谈的是 Open Time Card Mini 结合一些开源软件，来提供这种韧性。

> Thank you very much. I appreciate that introduction. I'm glad to be with you. I'm glad to be here with you this afternoon. Kudos to um Ahmad for all the pushing and the working that he's been working on with respect to pushing PTP. What I want to talk to you about today is mainly discussing [clears throat] extending on the topic of resiliency. I think it was a great two talks ago where they talked about resiliency, why you need resiliency, the importance of it. And they were talking about in particular the time card. In this particular case, I'm going to bring about some technology rather than time card. I'm going to talk about the open time card mini combined with some open source software to then provide the resiliency.

> **小结**　开场就把定位划清了：上一场讲的是"把时间卡做得更好"，这一场讲的是"把多台廉价机器组织起来"，前者是硬件路线，后者是系统路线。

02　数据中心为什么需要时间韧性



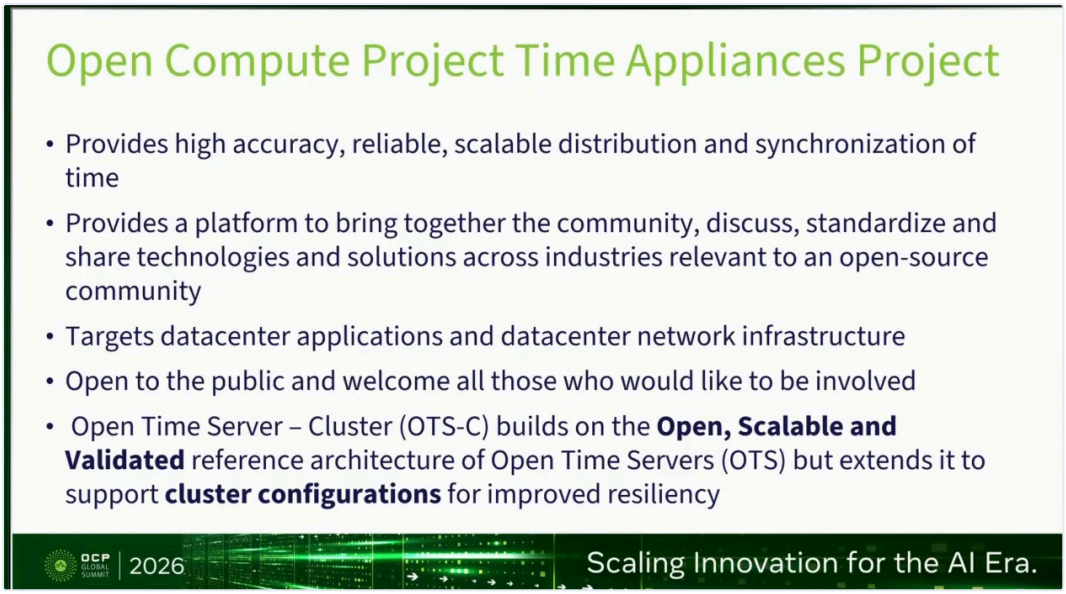
Need for Timing Resiliency at Data Centers：左侧列出安全、AI/5G/金融交易等时间敏感应用、数据完整性三点，右侧为全球数据中心分布图与美国 3897 个居首的统计表

那么，数据中心为什么需要时间韧性。我想我们大体上都明白为什么需要时间韧性，这非常重要。谈到时间韧性，从数据中心的角度可以看很多不同的应用，特别是涉及 AI、5G、金融交易的应用。所以动机是存在的，这里只是给一个概览，让大家知道我们确实需要时间韧性，而且有很多数据中心可以支撑这件事。但我这次主要想讲的，是在团队已经做出来的东西上继续往前推。Ahmad 做的、Julian 做的、以及他们这些年在 PTP 上开发的各种硬件……

> So the need for time resiliency with respect to data centers. I think we all sort of understand why we need the time resiliency. It's very important. When you talk about time resiliency, there are a lot of different applications that one can look at with respect to data centers. In particular, there's things involving applications for AI, 5G, financial trading. So the motivation is there and this is just an overview to just let you know sure, we definitely need time resiliency and there are a lot of data centers out there that can support that. Primarily I wanted to talk about though for this talk is extending on what pretty much the team has already done. For what Ahmad has done, Julian, all the

> **小结**　这一页的说服力不在文字而在右侧那张图：美国 3897 个数据中心、德国 464 个、英国 469 个——时间基础设施要覆盖的是这种量级的分布，而不是几十个机房。

03　OCP 时间设备项目（TAP）



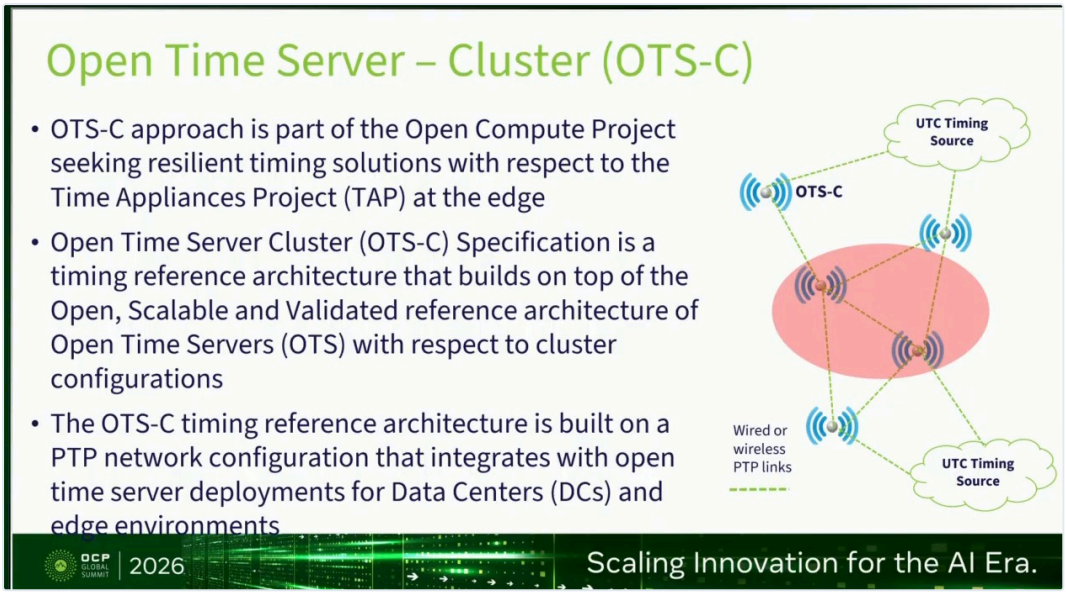
Open Compute Project Time Appliances Project：高精度可扩展的时间分发与同步、社区平台、面向数据中心应用、对公众开放、OTS-C 在 OTS 基础上扩展集群配置

……各种硬件。我想在这个基础上继续推进，利用一些有意思的概念往上搭。这里只是时间设备项目（Time Appliances Project）的概览。不熟悉这个项目的人可以看这一页，它总结了他们关心的事，也是我们作为一个群体关心的事：提供高精度、可靠、可扩展的时间分发与同步；提供一个平台把社区聚在一起，讨论、标准化并分享与开源社区相关的跨行业技术与方案；面向数据中心应用与数据中心网络基础设施；对公众开放，欢迎所有愿意参与的人。而这次演讲，我想谈的是在他们已经做好的开放时间服务器（Open Time Server）基础上继续——不是单台，而是开放时间服务器**集群**。在集群这个语境下，意思是：让我在一个地点或一个节点上拥有多个 grandmaster（主时钟），我能拿它们做什么？如果要去写一份规范，围绕它能发展出哪些概念？

> different kind of hardware that they've developed over the years with respect to PTP. I wanted to extend on that and leverage some some some interesting concepts that will build on that. So this is just an overview of the time appliances project. For those who don't know about the time appliances project this is gives you a summary of some of the things that they care about and what what we care about as a group. You know, providing high accuracy, reliable, scalable Um, know, distro and the synchronization of time, platform to bring together the community, um, you know, of course we're targeting data centers as far as the, uh, apps are concerned. Welcome to the public, welcome to, you know, those who want to be involved. And for this particular talk, I want to talk about building on what they've already done with respect to open time server. Instead, an open time server cluster. And in the cluster context, it's saying, let me have multiple, um, grandmasters, you know, at a at a at a place or at a node. What could I do with those? What are kind of the concepts that one could develop if one's if one was to develop a spec, um, that that that will actually be around that. [cough] So, this diagram just sort of gives an overview of what I mean by the OTS, uh,

小结　TAP 的既有成果是"一台机器做到高精度"，OTS-C 要回答的是"多台机器怎么协同"——从单点指标转向系统韧性，这是整个演讲的出发点。

04　OTS-C：把集群概念用在时间上



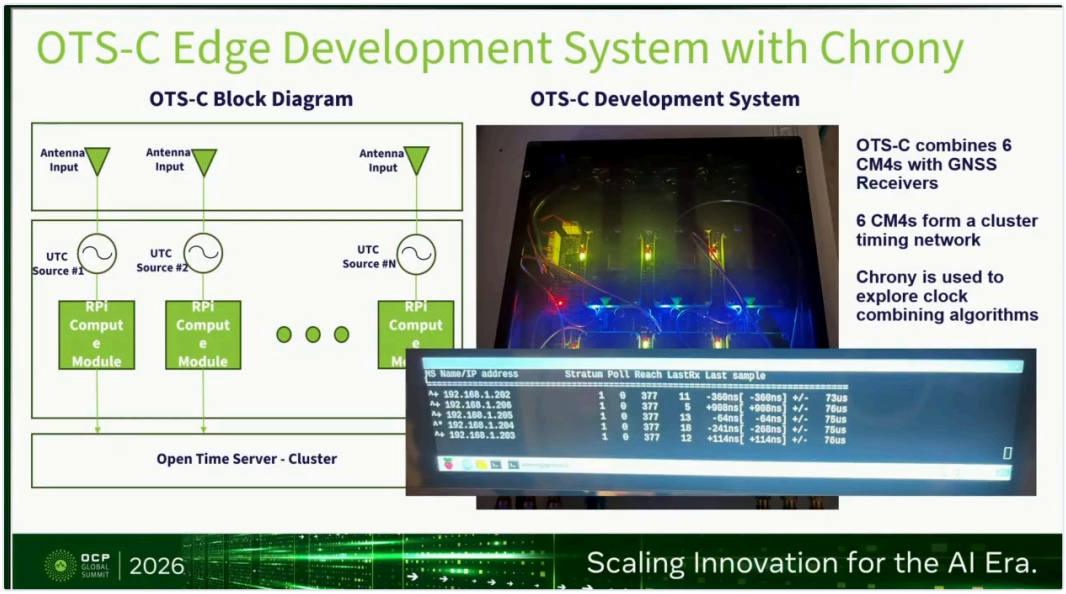
Open Time Server – Cluster (OTS-C)：左侧三条要点说明 OTS-C 定位与 PTP 网络架构，右侧为多台 OTS-C 经有线/无线 PTP 链路互联、上接 UTC 授时源的拓扑图

所以这张图大致给出了我说的 OTS-C 集群是什么意思。与其从分布式网络的角度去想——我们谈分布式授时的时候经常是不同节点——我这里谈的是一个**本地**的集群，就在站点本地。所以不是分布式的，而是我手上有多个单元、多台有算力的计算机。而且它不是为算力做集群，而是**为时间做集群**。这样一来，我就要谈时间系综（time ensemble）之类的东西了。如果我手上有多个时钟源、站点上有多个天线，我能怎么把它们组合起来？我能用什么独特的方式来做？这就是我想引出的问题：我该如何设计、如何开发一个支持这件事的集群架构。这里的一个关键假设是：即使我在不同地点没有时间，也总有一种把时间引进来的办法。所以我有多个时间源，可能是有线的也可能是无线的，但要确保我能把所有时间源都算进来，并且把它们利用起来，成为我集群的一部分。

> C cluster in terms of a cluster. Uh, rather than thinking about it from a distributed network, cuz a lot of times we talk about distributed timing and we have different nodes. What I'm talking about though is a cluster that's actually local, uh, to the site. So, rather than having it distributed, actually have multiple units, multiple computers that are available to me with compute power. And it's thinking about instead of clustering for the compute power, it's clustering clustering with respect to time. And with that, that means I want to actually now talk about time ensembles and things like that. What could I do if I have multiple clock sources, multiple antennas that are available at the site? How could I combine them? And what's what's a a unique way that I could do that? And so, this is what I'm sort of motivating or trying to get around is figuring out how I would design or develop a cluster architecture that would support that. And one of the key things here is I'm assuming that even though I don't have time at all different locations, um, there's always a way to get time in. So, I have multiple sources of time. And this could be either wireless or wired that's coming in, but making sure that I can account for all sources of time and leaving that, uh, or leveraging that that will be part of my, uh, cluster.

> **小结**　"为时间做集群"和"为算力做集群"的差别在于目标函数：前者优化的是时钟偏差与可用性，后者优化的是吞吐——所以它天然需要系综算法，而不是负载均衡。

05　原型：6 个 GNSS 接收机 + 6 张 OTC mini 卡



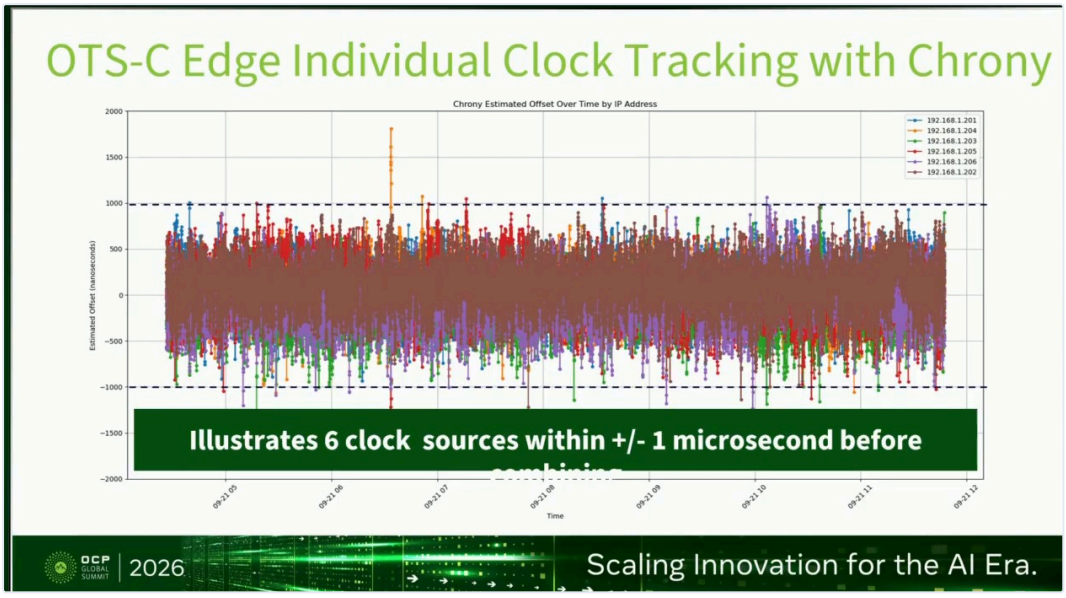
OTS-C Edge Development System with Chrony：左侧框图显示天线输入 → UTC 源 → RPi Compute Module → Open Time Server Cluster，右侧为实机照片与 Chrony 源列表

那我做了什么？我没有只是写一份规范、说"来，我们定义一份规范"。我一直喜欢动手做东西，喜欢做原型。所以我决定做的一件事是：做一个能代表 OTS-C 架构的示例系统原型。我做出来的东西我叫它 OTS-C 开发系统，它至少接入六个 GNSS 接收机、六个天线，并把它们与六张 Open Time Card Mini 组合起来。有了这些卡，我至少能做出一个初步概念：把架构与已有的硬件、再加上 Chrony 组合起来。我喜欢 Chrony 的原因是，Chrony 本身内置了把不同类型时间源汇总、组合起来的方法。所以想法是：如果我能拿现成的 Chrony 当作 NTP 合成器，再和我已有的硬件结合，至少能给出一个粗略的概念，说明 OTS-C 类型的集群能做成什么样。这是我初期集群的一张快照——在这个场景里，因为我有六个 CM4，我让其中一个 CM4 充当合成器，另外五个负责接入时间源。但如果我想用全部六个，我只要加一台远程计算机接进来，用那六个源一起做合成。

> So, what did I do? Well, instead of just coming up with a spec or saying, "Hey, let's just define a spec." I always like to create things. So, I like to develop prototypes. And one of the things I decided to do was, "Well, let's just create a prototype of a example system that will be representative of an OTSC type of architecture." And so, what I created is what I'm calling the OTSC development system um that pretty much brings in at least uh six GNSS receivers, six antennas, and I'm combining that with six open time card mini um cards. So, with those cards, [clears throat] I'm able to develop at least an initial concept of combining the the architecture with the hardware that's already there with Chrony. Um the reason why I like Chrony is because Chrony already has built in it uh ways of assembling or ways of actually bringing together different type of time sources. And so, the idea was if I could take Chrony that's already available uh as an NTP combiner and combine that with some existing hardware that I have, I could at least give a rough idea of what could be done from an OTC uh from from from an OTSC type of cluster uh concept. And this is just a snapshot that I had um of an initial cluster where at least for this particular case, since I had six uh CM uh 4s, uh in this particular scenario, I had one of the CM 4s that act as the combiner and the other five to actually bring in the sources. Uh but, if I wanted to use all six, I would just have a remote um computer that would actually be able to tie into this and then use those six sources to actually do the combining.

> **小结**　这套原型的成本结构值得注意：六块 CM4 加六张 OTC mini 卡，硬件全是 COTS（商用现货），真正稀缺的不是器件而是合成算法——这也是他反复强调"用 Chrony 起步"的原因。

06　单路跟踪：六路都在 ±1 微秒内



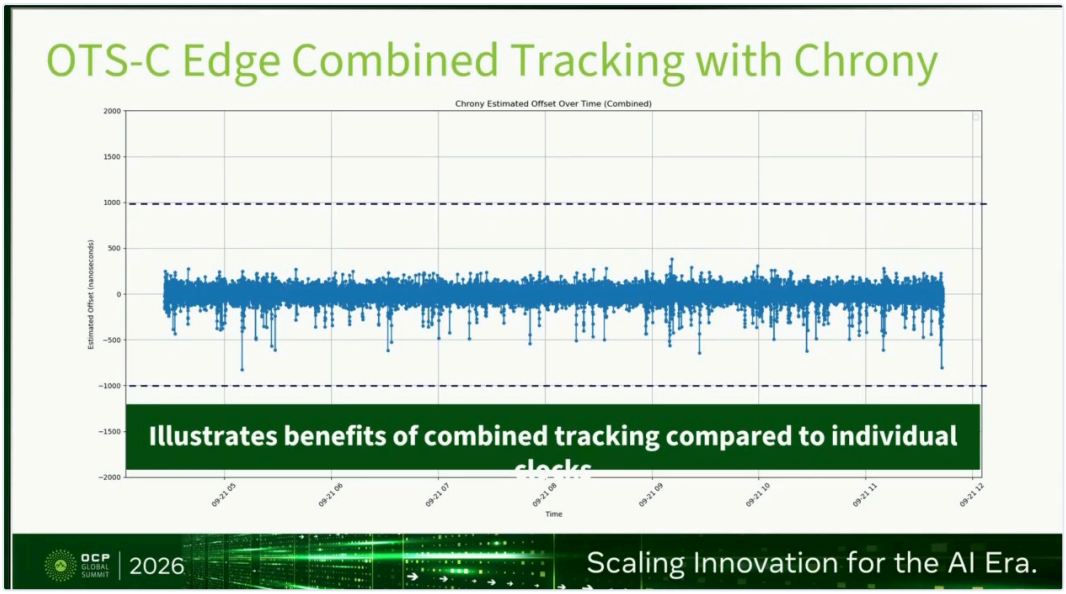
OTS-C Edge Individual Clock Tracking with Chrony：六条不同颜色的偏移曲线在 ±1000 纳秒带内交织，标题栏注明"6 个时钟源在合并前均处于 ±1 微秒以内"

这里是一些初步结果，给大家一个概念。前面我展示的那个原型有个有意思的地方：我用的是 DeskPi Super Six C 机箱。它本身没有内置 PTP 交换机，对吧？所以会有一些误差和问题，因为它并没有把精确时间传递出去。但我要特别说明：NTP 仍然可以消费硬件时间戳。这正是 Chrony 的强大之处之一——我利用的是我能拿到时间戳、NTP 能消费它们，进而把它们当作合成的来源。但在这个具体场景里，我只想展示：在这个时间源之上，我能在与 Chrony 合成之前拿到六个时钟源、并且都在正负一微秒以内。这张图展示了改进的效果，比 1 除以根号 n（这里是根号 6）还要稍好一点，因为我在 Chrony 内部还做了一些额外的滤波来进一步提升。

> So, here's some initial results that I wanted to provide to give you an idea of what it looks like. Now, interesting thing about that particular uh prototype that I showed you earlier, that's a Desk uh DeskPi Super Six C that I used uh to do that. That inherently does not have a PTP uh switch built in it, right? So, I'm going to have some errors and things associated with that because it's it's not it's not transferring the the precise time is there. But, I will want to I do want to make the note though that NTP can still consume hardware timestamps. And that's the one of powerful things about Chrony is I'm using the fact that I can get timestamps and NTP can can consume them and then potentially use that as of course a source of combining. But, in this particular case, I just want to show that over this source of time, I can get you know six clock sources within a plus or minus microsecond um before uh combining with Chrony. And this just shows the impact of the improvement that's there. A little bit better than one over square root of n, in this case square root of six. But, there is some additional filtering that I'm doing within Chrony to actually improve upon that. So, now with the combining and with the different kind of sources, at least I

> **小结**　±1 微秒是这套廉价方案的实测上限，而机箱没有 PTP 交换机意味着这个数字里还含着网络传输误差——换句话说，这是"最差配置"下的结果，不是优化后的结果。

07　合并跟踪：比 1/√n 更好



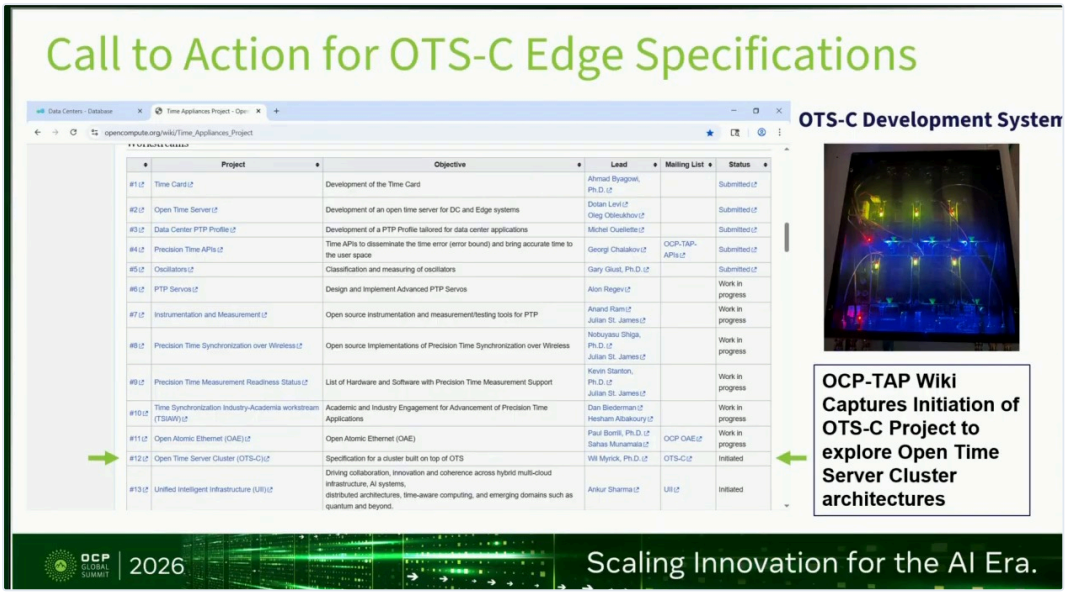
OTS-C Edge Combined Tracking with Chrony：合并后单一曲线收敛在 0 附近，标题栏注明"展示了合并跟踪相对单路跟踪的收益"

……至少我有了一个集群式构造能做成什么样的基线架构。这大概就是我这个项目想完成的事：把"如何围绕已有成果构建一个具体集群架构"的讨论启动起来。

> have a baseline architecture of what could be done in terms of a cluster type of construct. And that's what pretty much I was trying to get done for for for for for this particular project. And wanted to just sort of start the discussion of how one could then build a specific cluster architecture around what's already been done.

> **小结**　单路图是六条曲线各自抖动，合并图是一条曲线收敛——这张对比图才是整个原型的论据：六台廉价机器的合成结果优于任何单台，而且优于理论上的 1/√n。

08　行动号召：定义边缘规格



Call to Action for OTS-C Edge Specifications：OCP-TAP 项目清单截图，第 12 项 "Open Time Server Cluster (OTS-C)" 状态为 Initiated，Lead 为 Will Myrick

那么行动号召是什么？基本上就是把两场演讲之前你们听到的"韧性 PNT"（resilient PNT）概念再推高一层——推到我真的在本地站点上。想法是为这个架构定义**边缘特性**。你可以看到我们已经启动了，至少已经发起了一个工作组、一个工作流。再次感谢 Ahmad 帮我把它建起来。而这次演讲的主要目的，就是让大家知道：用非常便宜的 COTS 架构在授时上能做到什么；以及如果你把它看成一个开发系统——手上有多个天线、多个接收机——你可以怎么去设计出集群式的架构。

> So, what's the call for action? Pretty much is taking the concept you heard it earlier two talks ago about resilient PNT. This is just taking resilient PNT to another level where I'm actually at the local site. And the idea is to define the edge characteristics of this particular architecture. You can see that we've already started at least initiated a working group for that, a workflow. Again, thanks to Ahmad for helping me set that up. And primarily this talk was to just let everyone you know be aware of what can be done with very inexpensive COTS type of architectures with respect to timing. And then what could be combined if you think of it as a development system where I have multiple antennas and multiple receivers available. How could one potentially come up with the cluster type of architecture.

> **小结**　幻灯片上真正有信息量的是那张项目清单：第 12 项 OTS-C 状态仍是 Initiated，负责人是讲者本人——也就是说这项工作此刻还在"立项"阶段，没有规范草案。

09　谢谢与第一个问题：Chrony 怎么剔除坏钟



Thank You!

我的演讲就到这里，欢迎提问。

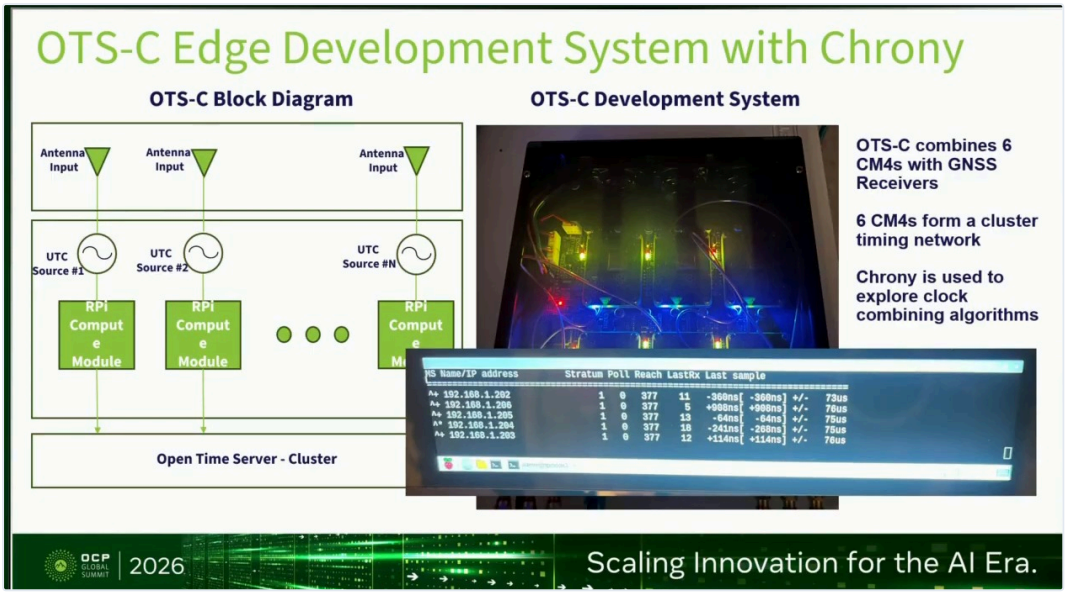
Nobu（NICT）：嗨，我叫 Nobu，来自 NICT。谢谢精彩的演讲。我对 Chrony 不太熟，但假设你有六个 GNSS 接收机，Chrony 怎么应对故障？比如其中一个 GNSS 接收机收不到信号、给出一个奇怪的时钟，Chrony 怎么从那五个健康的加一个坏的里面做出系综？

Wil Myrick：这是个好问题。Chrony 其实——如果你去看算法，你可以查 Chrony 的规范，看支撑它的底层算法。但它有一套非常具体的方式来判断某个信号或某个时钟源是否有效。它有一个规范，有一个误差容限，规定了哪些时钟源是可用的。如果你注意我刚才那个版本里的列表，我会回去看一下。对这一个，Chrony 有——注意那里有加号，然后还有一个星号，那意味着这些时钟都满足了 Chrony 的容限……

> So, that's it for my talk. Um, open for any questions. Hey. Hi, my name is Nobu from NICT. And thanks for the great talk. And I mean, I'm not so familiar with Chrony, but suppose you have six GNSS receivers. And how does Chrony works against malfunctioning? Like, for example, one of the GNSS receiver failed to receive the signal and give some weird clock. And how does the clock Chrony make the ensemble out of those five healthy ones plus one bad one? That's a great question. So, Chrony has actually a It's If you look at the algorithm, you can actually look up the specs on Chrony and see the underlying algorithms that support it. But, it has a very specific way of uh, determining whether or not that um, signal or that clock source is is going to be valid or not. So, it has a specification. It has an error um, where it has a tolerance about what clock sources are going to be available. If you notice the list for that particular um, um, version I had here, I'll go [music] back. Um, for this particular one, Chrony has it Notice that there's a plus plus pluses and then there's a There's an asterisk there. That means that those

> **小结**　这个回答其实承认了一件事：坏钟的剔除不是 OTS-C 自己设计的机制，而是 Chrony 已有的容限判定——原型阶段复用的是成熟软件的健壮性，而不是重新发明它。

10　问答续：日本的 cluster clock 项目与工作组节奏



OTS-C Edge Development System with Chrony（问答时切回该页）

……这些时钟都满足了 Chrony 的容限。所以接下来才做合成。它实际上会知道某个源是否超出容限，并且根本不允许它被纳入。所以这是使用 Chrony 的好处之一：它已经内置了这套东西。在此基础上，你想加多少源都可以加。但在这个具体场景里，我应该说明一点：它们全都是作为 NTP 服务器在运行，这样 Chrony 才能消费它们。

Nobu（NICT）：哦，好。这很棒，很让人印象深刻。关于你提到的这个子组，如果可以的话我想请求：如果你们打算开会，希望能安排成对日本友好的时间（笑）。因为 NICT 和大学里做控制理论的人合作做了一个项目，项目名叫 cluster clock，就是想从控制理论的角度去学习和研究这个算法。已经有一群人在做这件事，而且已经有很好的结果。所以我想，如果把他们也纳入进来会很好。

Wil Myrick：对，把他们纳入进来肯定很好。这也是我想做的事之一。这次演讲的一部分目的就是呼吁，这样我能把其他也在看这个题目的人聚集起来。

提问者：关于参与方式，我注意到日历上这个工作流还没有任何会议，有计划吗？

Wil Myrick：有计划。我还没建起来。我想做的其中一件事就是在这里启动它，看看有没有兴趣。

提问者：那下周？

Wil Myrick：下周？可以。

提问者：期待。

Wil Myrick：好，太好了。（笑）还有其他问题吗？还有谁？好，我们感谢 Myrick 博士。谢谢。

> clocks all met the tolerance for Chrony. So, then it would be the combining. It will actually know whether or not it is outside the tolerance and it won't even allow it to actually be included. So, that's one of the benefits of using Chrony. It already sort of has that built in. And then for that, you can add as many sources as you want to. But, it In this particular case, I should have uh, made the note that they were all acting as NTP servers uh, for Chrony to be able to actually consume it. Oh, well. Yeah, that's great. I mean, it impressive. And as for the uh, this subgroup that you mentioned, if I

> could ask request if you plan to have a meeting, it would be nice if you can make it to Japan friendly type. [laughter] Because NICT and university, I mean the control theory people have collaborated to make the I mean the project name is cluster clock. So just to make to learn and study about the algorithm from the control theory perspective. And so there are group of people who have been working on it. And they have great results already. So I mean I thought it I mean if you include them then it would be great. Yeah, it it would be great to definitely include them. That's one of the things I wanted to do. Part of this was to call out so I could gather gather the community of others that are also looking at this this topic. As far as getting involved, I noticed the calendar doesn't have any meetings for this work stream. Is there are there any plans? Yeah, there are plans. I have not set it up yet. I want to one of the things I wanted to do was sort of kick it off here to see if there's interest. So next week? Next week? Sure. Looking forward to it. Okay, great. [laughter] Any other questions? Anyone? All right. We'll thank Dr. Madnick. Thank you.

> **小结**　问答里最有价值的信息来自听众而非讲者：日本 NICT 与高校控制理论团队已经在做同名项目 cluster clock，说明"多源时钟系综"在学术侧早有积累，OCP 这边缺的是工程化与规范。

附录

附录 A　数据核对表

项目	数值 / 结论	来源	备注
演讲时长	11 分 4 秒（664.5 s）	视频元数据	—
原型构成	6 个 GNSS 接收机 + 6 个天线 + 6 张 Open Time Card Mini + 6 块 CM4	幻灯片 05 + 口播	实际用 1 块作合成器、5 块接入源
机箱型号	DeskPi Super Six C	口播	本身无内置 PTP 交换机
合成前精度	六路均在 ±1 微秒以内	幻灯片 06	图注原文 "within +/- 1 microsecond before combining"
合并后改善	略优于 1/√n（即 1/√6 ≈ 0.41）	口播	幻灯片未给出具体数值
合成软件	Chrony（作为 NTP 合成器）	幻灯片 05	六个源均以 NTP 服务器方式提供
数据中心数量参考	美国 3897、英国 469、德国 464、加拿大 469	幻灯片 02	来源标注 datacentermap.com
OTS-C 项目状态	Initiated（已发起）	幻灯片 08	OCP-TAP 项目清单第 12 项

项目	数值 / 结论	来源	备注
工作组会议	尚未排期，讲者称"下周"启动	口播（问答）	截至演讲时日历为空

附录 B　时间轴

时间	内容
00:00	开场，致谢 Ahmad，说明本次不谈时间卡而谈技术组合
00:50	数据中心为什么需要时间韧性：安全、AI/5G/金融、数据完整性
01:20	TAP 项目概览与 OTS-C 的定位
02:33	OTS-C 拓扑：本地集群而非分布式，多源合成
03:47	原型：6 GNSS + 6 OTC mini + 6 CM4，用 Chrony 做合成
05:22	单路跟踪结果：±1 微秒；说明机箱无 PTP 交换机
06:19	合并跟踪结果：优于 1/√n
06:54	行动号召：定义边缘特性，已发起工作组
07:46	提问：Chrony 如何剔除故障时钟源
09:00	追问：日本 cluster clock 项目、工作组会议排期
10:49	结束

附录 C　术语表

缩写	全称	说明
OTS	Open Time Server	TAP 定义的开放时间服务器参考架构
OTS-C	Open Time Server – Cluster	在 OTS 基础上扩展的集群配置
TAP	Time Appliances Project	OCP 的时间设备项目
PTP	Precision Time Protocol	IEEE 1588 精确时间协议
NTP	Network Time Protocol	网络时间协议

缩写	全称	说明
GNSS	Global Navigation Satellite System	全球导航卫星系统
UTC	Coordinated Universal Time	协调世界时
CM4	Raspberry Pi Compute Module 4	树莓派计算模块 4
COTS	Commercial Off-The-Shelf	商用现货
PNT	Positioning, Navigation and Timing	定位、导航与授时
Chrony	—	开源 NTP 实现，内置多源选择与合成